DHBW
Duale Hochschule
Baden-Württemberg
Mosbach

Dokumentation der Projektarbeit

Projekt: Stock Subscription Service

von

John Hohn, Simon Hopfhauer, Tilman Voit

Inhaltsverzeichnis

- Organisatorisches
  - Gruppenmitglieder
  - Projektplan
  - Aufgabenverteilung
  - Verwendete Technologien
  - Verwendete APIs
- Grundidee der Web-Applikation
- Beschreibung der verwendeten Services
- Gesamtprozess
- Hilfsprozess: Aktienkursabfrage
- Prozess: Aktie abonnieren
- Prozess: Automatische Kursüberwachung
- Prozess: Wärungsumrechnung
- Prozess: Abonnement kündigen

Organisatorisches

Gruppenmitglieder

Name	Matrikelnummer
John Hohn	4692407
Simon Hopfhauer	6092891
Tilman Voit	5136540

Projektplan

06.02.26	Ideenfindung/Projekt-Start
11.02.2026	Erstellung BPMN-Diagramme
13.02.2026	Besprechung zu BPMN-Diagrammen
27.02.2026	Fertigstellung: Implementierung
04.03.2026	Abschlussbesprechung
10.03.2026	Fertigstellung: Präsentation

Aufgabenverteilung

Aufgabe	Teammitglied
BPMN:	

Aufgabe	Teammitglied
Aktie abonnieren	Simon Hopfhauer
Aktienkursabfrage	John Hohn
Automatische Kursüberwachung	Tilman Voit
Währungsumrechnung	Simon Hopfhauer
Abonnement kündigen	John Hohn
Implementation:	
Auswahl und Analyse externer APIs	Tilman Voit
Subscription-Service	Simon Hopfhauer
Composite-Service (StockSubscriptionService)	John Hohn
Alert-Logik	Tilman Voit
Fehlerbehandlung	Simon Hopfhauer
Testen mit SOAP-UI	Tilman Voit
Dokumentation:	
Architektur-Beschreibung	John Hohn
Service-Komposition	Simon Hopfhauer
OPENAPI	Tilman Voit
HTTP-Nachrichten	Tilman Voit
UI:	
Eingeben der Daten	Tilman Voit
Anzeigen der Ergebnisse	John Hohn

Verwendete Technologien

- Quarkus
- REST
- JSON (Datenformat)
- Docker
- PostGreSQL

Verwendete APIs

- CurrencyService: https://cdn.jsdelivr.net/npm/@fawazahmed0/currency-api@latest/v1/currencies/usd.json
- StockService: https://www.alphavantage.co/

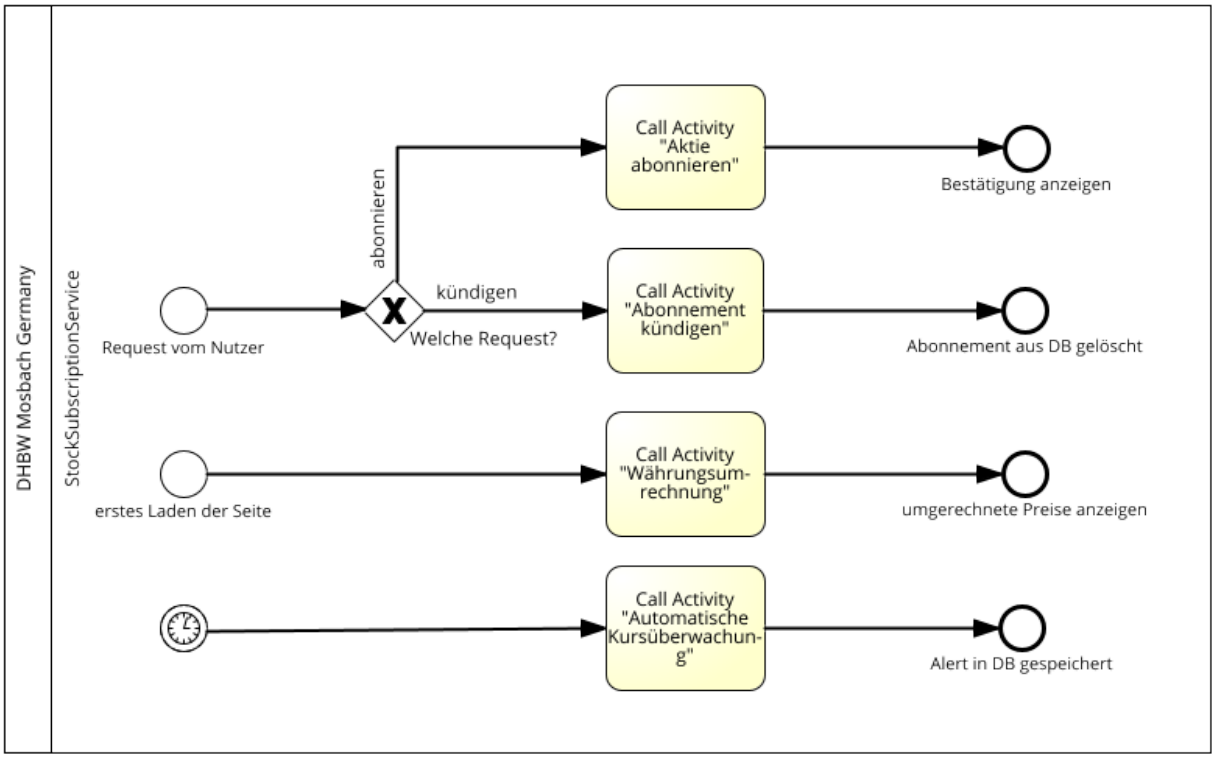
Grundidee der Web-Applikation

Die Web-Applikation „StockSubscriptionService“ soll es Nutzern ermöglichen, Aktien gezielt zu abonnieren und jederzeit aktuelle Kursinformationen einzusehen. Zusätzlich können Nutzer optional Benachrichtigungen erhalten, wenn eine Aktie einen bestimmten Schwellenwert erreicht oder es größere Kursänderungen gibt. Das Ziel ist, eine einfache, aber funktionale Plattform zu schaffen, auf der man mehrere Aktien gleichzeitig verfolgen kann, ohne dass ein komplexes Login-System notwendig ist. Die Eingabe der Aktien erfolgt direkt über den Namen oder das Symbol der Aktie, wodurch die Bedienung möglichst unkompliziert bleibt. Technisch betrachtet soll die Applikation mehrere Services kombinieren, um die einzelnen Aufgaben zu übernehmen, sodass der Hauptservice selbst nur die Orchestrierung der Abläufe übernimmt. Dadurch lässt sich der Prozess übersichtlich gestalten, und die einzelnen Services können unabhängig voneinander entwickelt, getestet und ggf. ausgetauscht werden. Auf diese Weise wird die Grundidee der Anwendung, aktuelle Aktieninformationen schnell und zuverlässig bereitzustellen und Nutzer über relevante Kursentwicklungen zu informieren, klar abgebildet.

Beschreibung der verwendeten Services

Die Web-Applikation basiert auf mehreren Web-Services, die jeweils spezielle Aufgaben übernehmen und gemeinsam den „StockSubscriptionService“ als Composite-Service bilden. Der **StockSubscriptionService** ist der zentrale Service, der die verschiedenen Services aufruft und die Daten zusammenführt. Der **Alert-Service** prüft dabei, ob eine große Kursveränderung im Vergleich zum letzten Datenpunkt vorliegt, und erzeugt bei Bedarf eine Benachrichtigung. Der **Subscription-Service** verwaltet, welche Aktien von welchen Nutzern abonniert wurden, speichert diese Informationen und stellt sie für die Kursabfrage bereit. Für die aktuellen Kursdaten wird ein externer **StockService** verwendet, der über öffentliche APIs wie Alpha Vantage oder Finnhub die aktuellen Aktienkurse bereitstellt. Falls der Nutzer die Kurse in einer anderen Währung angezeigt bekommen möchte, greift die Applikation zusätzlich auf einen externen **CurrencyService** zurück, der die Umrechnung übernimmt. Durch diese Kombination aus internen und externen Services entsteht ein flexibles System, das sowohl die Orchestrierung der Daten übernimmt als auch auf externe, aktuelle Informationen zugreift, während Alerts und Abos unabhängig davon verwaltet werden. So wird die Funktionalität der Applikation modular und nachvollziehbar umgesetzt.

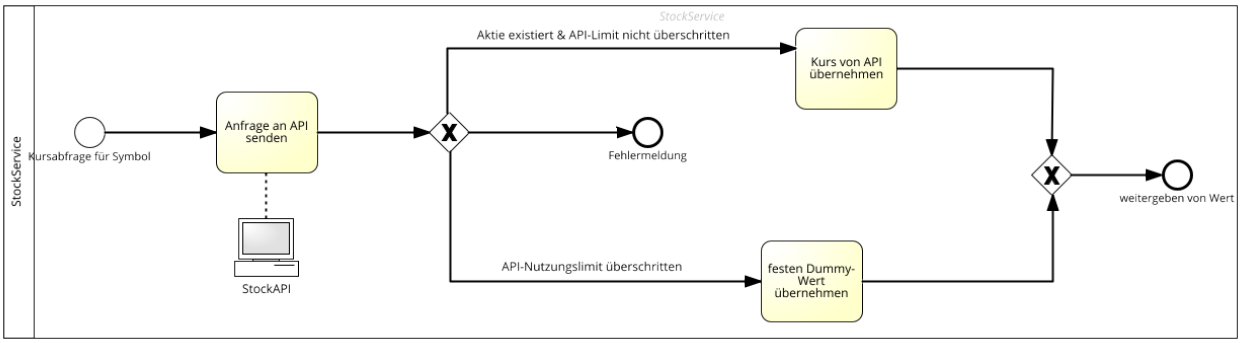
Gesamtprozess



Der dargestellte Gesamtprozess beschreibt die Orchestrierung des StockSubscriptionService als zentralen Composite Service. Der Ablauf kann entweder durch eine Benutzeranfrage oder durch ein zeitgesteuertes Ereignis gestartet werden. Im Fall einer Benutzerinteraktion beginnt der Prozess mit einem Request vom Nutzer. Über ein exklusives Gateway wird entschieden, welche Art von Anfrage vorliegt. Möchte der Nutzer eine Aktie abonnieren, wird die entsprechende Call Activity „Aktie abonnieren“ aufgerufen. Nach erfolgreicher Durchführung erhält der Nutzer eine Bestätigung. Handelt es sich hingegen um eine Kursabfrage, wird die Call Activity „Aktienkursabfrage“ gestartet, woraufhin der aktuelle Kurs berechnet und angezeigt wird.

Unabhängig davon existiert ein zweiter Startpunkt in Form eines Timer-Events. Dieses löst regelmäßig die Call Activity „Automatische Kursüberwachung“ aus. Dabei werden abonnierte Aktien überprüft und gegebenenfalls Benachrichtigungen erzeugt. Durch die Verwendung von Call Activities wird der Gesamtprozess übersichtlich gehalten, während die detaillierte Logik in separaten Teilprozessen gekapselt ist. Somit fungiert der StockSubscriptionService als Orchestrator, der unterschiedliche Abläufe koordiniert und externe sowie interne Services strukturiert einbindet.

Hilfsprozess: Aktienkursabfrage



Dieser Prozess beschreibt die Abfrage aktueller Aktienkurse durch den Benutzer. Der Benutzer gibt die gewünschte Aktie ein, woraufhin der StockSubscriptionService zunächst die Eingabe validiert. Ungültige Eingaben führen zu einer Fehlermeldung und einem Abbruch des Vorgangs. Nach erfolgreicher Validierung wird der StockService aufgerufen, um den aktuellen Kurs in US-Dollar abzurufen. Es wird im Anschluss der CurrencyService aufgerufen, der den Kurs in die gewünschte Währung umrechnet. Abschließend zeigt der StockSubscriptionService den umgerechneten Kurs dem Benutzer an. Der Prozess enthält somit sowohl die Interaktion mit externen Services als auch die Verarbeitung und Aggregation der Daten für die Anzeige. Durch die explizite Validierung der Eingaben und die Fehlerbehandlung ist gewährleistet, dass der Benutzer nur korrekte Informationen erhält.

HHTP-Nachricht:

Abfrage Aktienkurs:

Curl

```
curl -X 'GET' \
  'http://localhost:8083/stock/quote?symbol=AAPL' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8083/stock/quote?symbol=AAPL
```

Server response

Code	Details
200	Response body

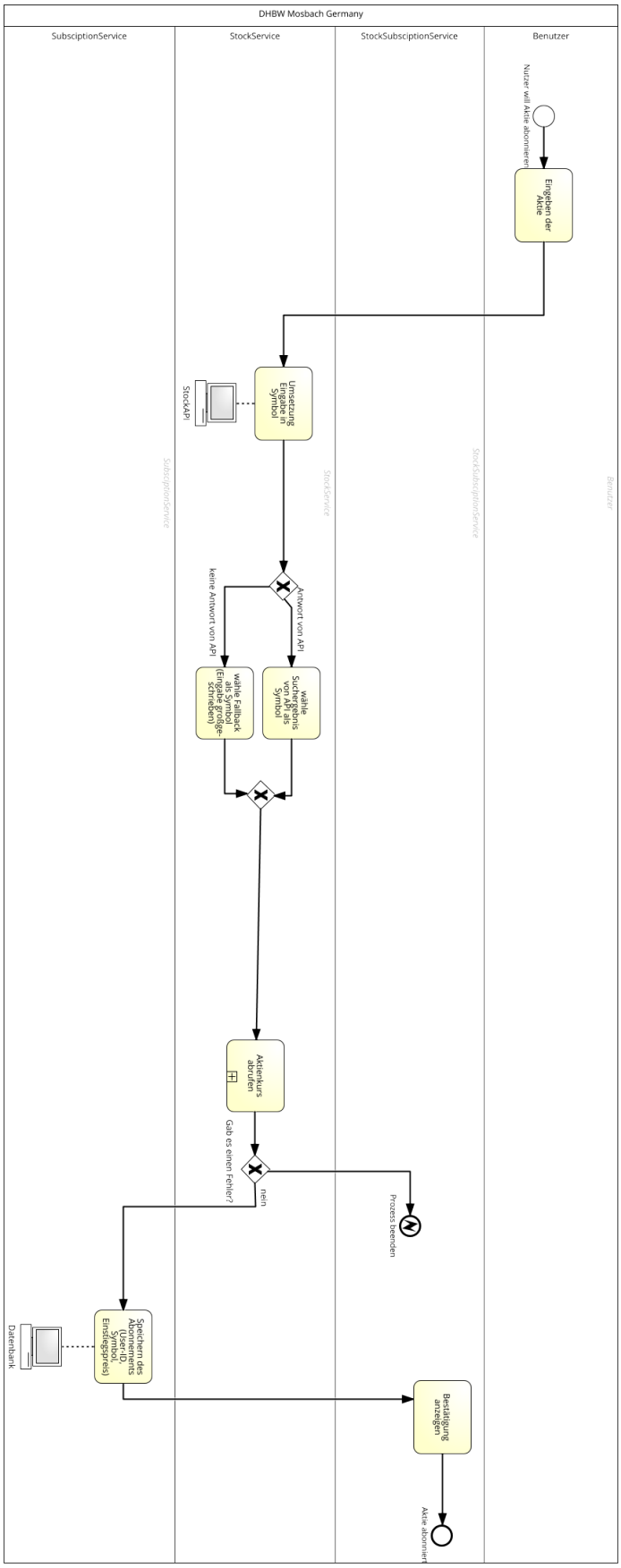
```
{
  "symbol": "AAPL",
  "priceUsd": 260.83
}
```

Download

Response headers

```
content-length: 35
content-type: application/json;charset=UTF-8
```

Prozess: Aktie abonnieren



Der Prozess „Aktie abonnieren“ beschreibt, wie ein Nutzer eine Aktie für die Beobachtung auswählt und in das System aufnimmt. Der Ablauf beginnt, sobald der Benutzer die gewünschte Aktie eingibt. Zunächst wird die Eingabe auf zulässige Zeichen und Länge überprüft, um sicherzustellen, dass Sonderzeichen oder unvollständige Daten nicht zu fehlerhaften Anfragen führen. Anschließend prüft der Service, ob eine gültige User-ID vorhanden ist. Fehlt diese, wird der Vorgang mit einer entsprechenden Fehlermeldung abgebrochen. Danach ruft der StockSubscriptionService den externen StockService auf, um die Existenz der Aktie zu bestätigen. Falls die Aktie nicht existiert, wird ebenfalls eine Fehlermeldung ausgegeben und der Prozess abgebrochen. Wenn alle Validierungen erfolgreich sind, wird die Abonnement-Information über den Subscription-Service gespeichert. Abschließend erhält der Benutzer eine Bestätigung, dass die Aktie erfolgreich abonniert wurde. Fehlerpfade wie ungültige Eingabe oder nicht vorhandene Aktie sind explizit vorgesehen, sodass der Prozess robust gegenüber fehlerhaften Eingaben bleibt. Die klare Trennung zwischen Benutzerinteraktion, Validierung, externem Service und Speicherung sorgt dafür, dass die Verantwortung jedes Services nachvollziehbar bleibt.

HTTP-Nachrichten:

Aktien-Symbol suchen:

Curl

```
curl -X 'GET' \
  'http://localhost:8083/stock/resolve?query=Apple' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8083/stock/resolve?query=Apple
```

Server response

Code	Details
200	Response body

```
{
  "query": "Apple",
  "resolvedSymbol": "APLE"
}
```

Download

Response headers

```
content-length: 41
content-type: application/json;charset=UTF-8
```

Abonnement in SubscriptionService speichern:

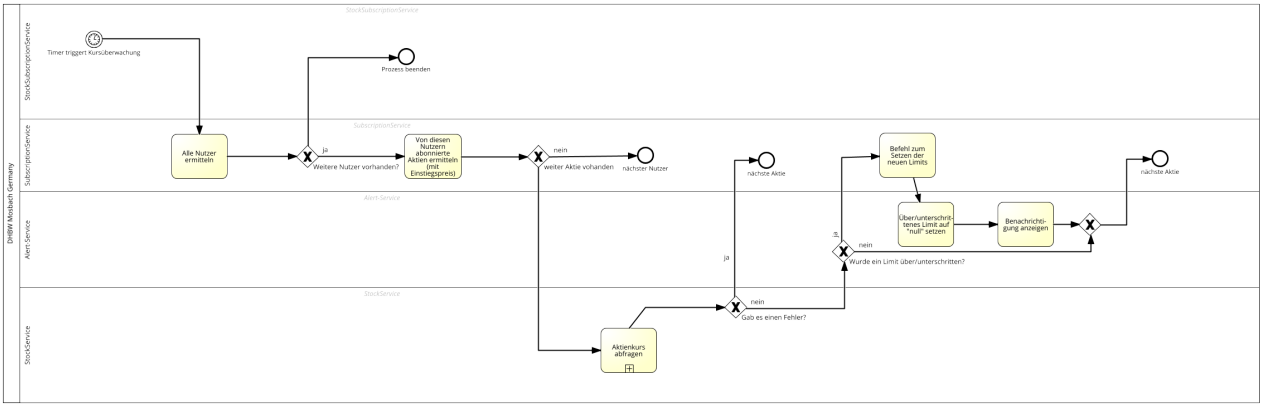
Curl

```
curl -X 'PUT' \
  'http://localhost:8081/subscriptions/test/AAPL' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "upperThresholdUsd": 0,
  "lowerThresholdUsd": 0,
  "currentPriceUsd": 150
}'
```

Request URL

```
http://localhost:8081/subscriptions/test/AAPL
```

Prozess: Automatische Kursüberwachung



Der Prozess der automatischen Kursüberwachung dient dazu, bei größeren Veränderungen der Aktienkurse Benachrichtigungen an die Nutzer zu generieren. Der StockSubscriptionService wird durch ein Timer-Event getriggert. Zunächst prüft der Service, ob die User-ID vorhanden ist; falls nicht, wird der Vorgang abgebrochen. Danach ermittelt der Subscription-Service die abonnierten Aktien des jeweiligen Nutzers. Für jede Aktie wird der aktuelle Kurs über den StockService abgefragt. Anschließend prüft der Alert-Service, ob der Kurs eine festgelegte Schwelle überschreitet oder eine starke Veränderung im Vergleich zum letzten Datenpunkt stattgefunden hat. Falls eine Benachrichtigung notwendig ist, wird diese generiert und dem Benutzer bereitgestellt. Der Prozess endet, wenn alle Aktien geprüft wurden und ggf. Alerts versendet wurden. Durch diesen Ablauf werden wiederkehrende Prüfungen automatisiert und Nutzer rechtzeitig über wichtige Kursbewegungen informiert, während die Trennung der Services die Wartbarkeit des Systems erhöht.

HTTP-Nachrichten:

Alle Nutzer ermitteln:

Curl

```
curl -X 'GET' \
  'http://localhost:8081/subscriptions' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8081/subscriptions
```

Server response

Code	Details
200	**Response body**

```
{}
```

Download

Response headers

```
content-length: 2
content-type: application/json;charset=UTF-8
```

Abonnements für Nutzer erhalten:

Curl

```
curl -X 'GET' \
  'http://localhost:8081/subscriptions/test' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8081/subscriptions/test
```

Server response

Code	Details
200	Response body

```
{
  "userId": "test",
  "entries": []
}
```

Download

Response headers

```
content-length: 30
content-type: application/json;charset=UTF-8
```

Prüfung auf über-/unterschreiten von Grenzen:

Curl

```
curl -X 'POST' \
  'http://localhost:8082/alerts/evaluate' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "userId": "string",
  "symbol": "string",
  "currentPriceUsd": 100,
  "upperThresholdUsd": 90,
  "lowerThresholdUsd": 80
}'
```

Request URL

```
http://localhost:8082/alerts/evaluate
```

Server response

Code	Details
200	Response body

```
{
  "notify": true,
  "alerts": [
    {
      "id": 1,
      "userId": "string",
      "symbol": "STRING",
      "previousPriceUsd": 100,
      "currentPriceUsd": 100,
      "percentChange": 0,
      "message": "OBERES LIMIT ERREICHT: STRING hat den oberen Schwellenwert erreicht!",
      "createdAt": "2026-03-11T12:54:38.6838604097"
    }
  ]
}
```

Download

Response headers

```
content-length: 266
content-type: application/json;charset=UTF-8
```

Ober-/Untergrenze ändern:

Curl

```
curl -X 'PUT' \
  'http://localhost:8081/subscriptions/test/AAPL/reset' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "upperThresholdUsd": 10,
  "lowerThresholdUsd": 100
}'
```

Request URL

```
http://localhost:8081/subscriptions/test/AAPL/reset
```

Server response

Code	Details
200	Response body

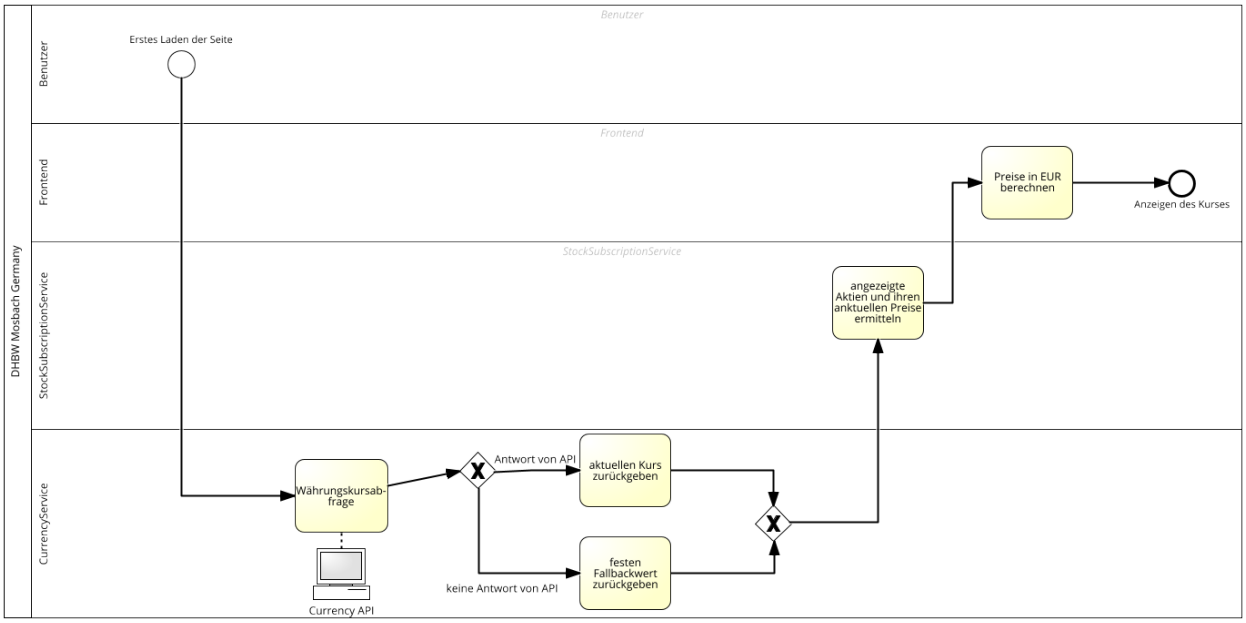
```
{
  "status": "reset"
}
```

Download

Response headers

```
content-length: 18
content-type: application/json;charset=UTF-8
```

Prozess: Wärungsumrechnung



HTTP-Nachricht:

Währungsumrechnung:

Curl

```
curl -X 'GET' \
  'http://localhost:8084/currency/convert?amountUsd=100&to=EUR' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8084/currency/convert?amountUsd=100&to=EUR
```

Server response

Code	Details
200	**Response body**

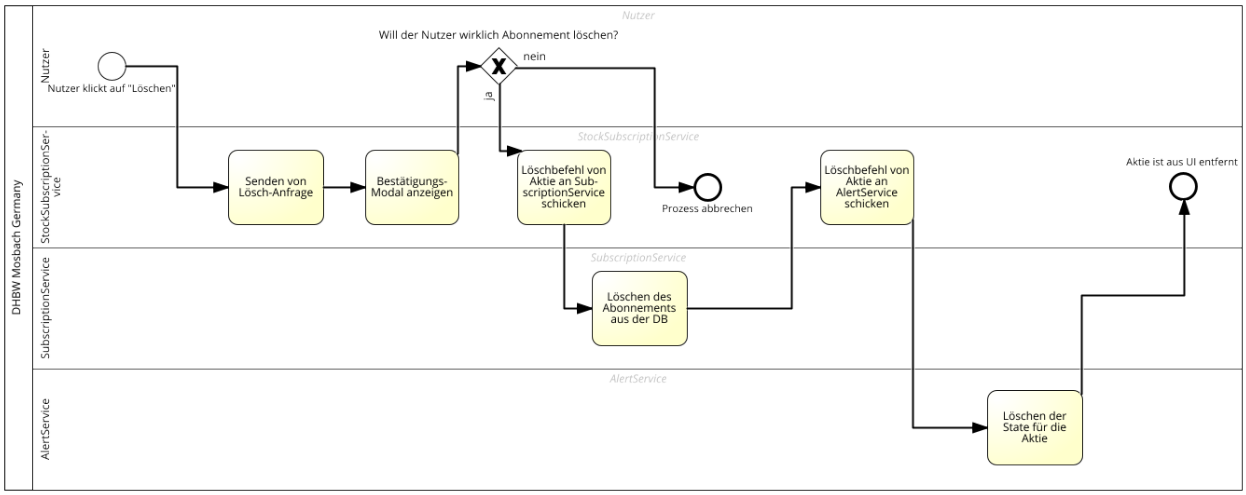
```
{
  "from": "usd",
  "rate": 0.86020534,
  "convertedAmount": 86.020534,
  "to": "eur"
}
```

Download

Response headers

```
content-length: 71
content-type: application/json;charset=UTF-8
```

Prozess: Abonnement kündigen



HTTP-Nachrichten:

Abo aus SubscriptionService löschen:

Curl

```
curl -X 'DELETE' \
  'http://localhost:8081/subscriptions/test/AAPL' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8081/subscriptions/test/AAPL
```

State für Aktie in AlertService löschen:

Curl

```
curl -X 'DELETE' \
  'http://localhost:8082/alerts/state/test/AAPL' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8082/alerts/state/test/AAPL
```

Server response

Code	Details
204	

Openapi-Dateien:

Die Openapi-Dateien sind außerhalb der Dokumentation in den folgenden Dateien zu finden:

- SubscriptionService: Openapi-SubscriptionService.txt
- AlertService: Openapi-AlertService.txt
- StockService: Openapi-StockService.txt
- CurrencyService: Openapi-CurrencyService.txt